statistics.py

`src/moe_pruning/scoring/statistics.py · 175 lines · module moe_pruning.scoring.statistics`

What it does

Streams per-layer, per-expert statistics from the instrumented forward passes into float64 running sums. Everything needed for scoring and bias initialisation comes from these sums; no per-token data is kept.

How it works

- `ExpertStatisticsAccumulator.update()` takes one `MoEForwardRecord` plus a real-token mask and adds counts, gate sums, output norms, token contributions, importance and the weighted output sums used by the bias formula.
- Bias-initialisation sums use only the calibration training split (`bias_mask`), keeping validation unseen.
- Works for top-1 and top-k routing; computation happens on the model device, accumulation on CPU in float64.
- `LayerStatistics` stores and restores the sums (`expert_stats.pt`).

Defined in this file

Extracted from the source code (classes, public functions and methods).

Name	Description
`class LayerStatistics`	Float64 running sums of one layer, per expert and per layer.
`.observed() [property]`	Which experts received at least one real token.
`.to_state()`	Dictionary for torch.save.
`.from_state(state)`	Restore from the saved dictionary.
`class ExpertStatisticsAccumulator`	Collects LayerStatistics for many layers from forward records.
`.update(record, token_mask, bias_mask)`	Add one forward record.
`.state()`	Saved form of all layers' statistics.
`load_statistics(state)`	Restore LayerStatistics objects from expert_stats.pt.

Connections

Uses: `moe_pruning.model.architecture`, `moe_pruning.scoring.easyep`

Used by: `scripts/collect_stats.py`, `scripts/replace_experts.py`, `scripts/score_experts.py`, `src/moe_pruning/pruning/bias_initialization.py`, `src/moe_pruning/pruning/expert_selection.py`